



UNIVERSITY OF BUEA
FACULTY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF COMPUTER ENGINEERING

SYSTEM AND NETWORK PROGRAMMING
CEF488

Project Topic: Distributed File Sharing System

GROUP MEMBERS

NAME	MATRICULE	SIGNATURE
EFUETNGONG GENESIS TAZISONG	FE23A041	
NEIL AMBAYIH	FE23A105	
TAKU YVETTE WUTOH	FE23A152	
TAMANTENG ABERLINE NGUEMUTANG	FE23A154	

Course Instructor
Mr. FORCHA GLEN

Declaration of Originality

This submission is entirely our own work. No material from other sources has been used without proper acknowledgment.

Table of Contents

HONESTY REPORT	3
1. Abstract:	4
2. Introduction:	5
3. Project Objectives:	6
4. Methodology:	7
High-Level Design Decisions	7
Code Structure	7
5. Result:	8
Verification using terminals as nodes	8
Distributed Network Setup	9
6. Analysis and discussion:	11
7. Challenges faced and solution	12
Mismatched Expected Hashes	12
Solution	12
VirtualBox connection refused Error	12
Solution	12
8. References	13

HONESTY REPORT

TASK	RESPONSIBLE MEMBER(S)	CONTRIBUTOR(S)	REVIEWER(S)
Project overview	All	All	All
Implementation	All	All	All
Testing and debugging	All	All	All
Report writing	All	All	All

1. Abstract:

This lab covers the creation and live setup of a secure, multi-threaded file-sharing program written in C. The main milestone was moving the program off a single computer and running it across two separate machines connected via a personal computer's hotspot. The server uses TCP sockets for reliable file transfers and POSIX threads to handle multiple clients at the same time.

For security, a simple user login system and an inline XOR file encryption cipher were added. To ensure files do not get corrupted during transmission, the server decrypts files on the fly into a temporary file to calculate a 32-bit integrity checksum before sending them. Testing was completed successfully by bridging an Ubuntu VirtualBox VM directly to a physical host laptop over the wireless hotspot network, resulting in flawless authentication, file sharing, and matching file hashes.

2. Introduction:

In network programming, running an application on a single local computer (localhost) is very different from deploying it across multiple physical machines. When data moves between separate computers, developers can no longer rely on internal system safety. Instead, they must actively handle dynamic network routing, virtual machine network isolation, and the security risks of sending data over a shared network link. This project addresses these practical challenges by transforming a basic file-sharing program into a secure, distributed, multi-threaded client-server system. The system was successfully set up and tested across two separate computers connected to a shared personal computer's hotspot network.

To allow multiple clients to connect at the same time, the server uses concurrent multi-threading pathways. To handle the security risks of using a shared mobile network, two key features were added to the system architecture: a strict user login validation gate and an inline end-to-end XOR stream encryption cipher. Additionally, to protect files from corruption or tampering during transmission, an on-the-fly 32-bit checksum verification loop was integrated into the file transfer pipeline. This section sets up the context for the system design, components used, and testing results discussed in this report.

3. Project Objectives:

- To move a network program from a local computer (localhost) to a real distributed network using a wireless hotspot (from a pc).
- To handle multiple clients simultaneously using POSIX multi-threading (pthread).
- To build a custom application protocol with fixed headers to send commands and files without data mix-ups.
- To protect data privacy and check for file corruption using stream encryption and checksum verification.
- To configure virtual machine network settings using a Bridged Adapter to allow real network communication.

4. Methodology:

High-Level Design Decisions

- **Multi-Threading (Thread-per-Client):** POSIX threads (pthread) were chosen so the server can talk to multiple clients at once. Using detached threads ensures that the operating system automatically cleans up memory as soon as a client disconnects.
- **TCP Sockets (SOCK_STREAM):** TCP was selected instead of UDP because files cannot tolerate missing bytes. TCP guarantees that data arrives complete and in the right order.
- **User Access Control:** Which is an added feature, a simple login system with a built-in username and password database was added. Clients must authenticate successfully before they are allowed to upload or download files. Unrecognized users are disconnected immediately.
- **XOR Stream Encryption:** To keep files private across the shared hotspot network, an inline XOR cipher alters the file bytes during transmission using a looping key and variable offset.
- **On-the-Fly Checksum Sync:** Which is an added feature, to check for file corruption, the server decrypts files into a quick temporary file (temp_dec_[client_fd].txt) to calculate a 32-bit checksum. The temporary plaintext file is immediately erased using unlink() for security, and the hash is sent to the client to confirm a perfect file match.

Code Structure

- **protocol.h:** Defines common settings, the login layout, and the main packet header (struct msg_header) which carries commands, file sizes, and the file verification hash.
- **network_utils.c:** Contains helper functions like send_all() and recv_all() to handle split network packets, apply_xor_cipher() for encryption, and calculate_file_checksum().
- **server.c:** The main server program. It listens for incoming connections, checks user logins, manages client threads, and processes file uploads and downloads.
- **client.c:** The user terminal program. It connects to the server's IP address, logs the user in, handles local file decryption, and verifies that file hashes match.

5. Result:

Verification using terminals as nodes

- Output of make showing successful compilation of files.

```
yvette@yvette-Latitude-7400-2-in-1:~/project_sys_net_prog$ make
gcc -Wall -Wextra -g -O2 server.c network_utils.c -o file_server
gcc -Wall -Wextra -g -O2 client.c network_utils.c -o file_client
yvette@yvette-Latitude-7400-2-in-1:~/project_sys_net_prog$
```

- Server started and running.

```
yvette@yvette-Latitude-7400-2-in-1: ~/project_sys_net_prog
yvette@yvette-Latitude-7400-2-in-1:~/project_sys_net_prog$ ./file_server
[Server] Secure environment running on port 8080...
```

- Clients running.

```
yvette@yvette-Latitude-7400-2-in-1:~/project_sys_net_prog$ ./file_client
[Client] Authentication Verification Complete. Session Secure.
```

```
=====
DISTRIBUTED FILE SYSTEM MENU
=====
1. Upload a File
2. Download a File
3. Exit System
-----
Enter your choice (1-3):
```

- Successful upload of a file by a client

```
=====
DISTRIBUTED FILE SYSTEM MENU
=====
1. Upload a File
2. Download a File
3. Exit System
-----
Enter your choice (1-3): 1
Enter local filename to UPLOAD: report.txt
[Client] Server safely accepted and verified file integrity.
```

- Successful download of a file by a client

```
yvette@yvette-Latitude-7400-2-in-1:~/project_sys_net_prog$ ./file_client
[Client] Authentication Verification Complete. Session Secure.

=====
DISTRIBUTED FILE SYSTEM MENU
=====
1. Upload a File
2. Download a File
3. Exit System
-----
Enter your choice (1-3): 2
Enter filename to DOWNLOAD: neil.txt
[Client] Success! File downloaded completely and decrypted.
[Client] Verified Integrity Hash matching: 1DDA040D

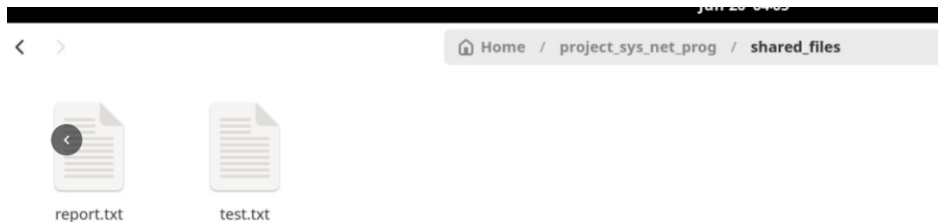
=====
DISTRIBUTED FILE SYSTEM MENU
=====
1. Upload a File
2. Download a File
3. Exit System
-----
Enter your choice (1-3): 3
Exiting execution stream loop...
yvette@yvette-Latitude-7400-2-in-1:~/project_sys_net_prog$ cat neil.txt

hi i am Neil
yvette@yvette-Latitude-7400-2-in-1:~/project_sys_net_prog$
```


- logs of concurrent events happening in the server

```
yvette@yvette-Latitude-7400-2-in-1:~/project_sys_net_prog$ ./file_server
[Server] Secure environment running on port 8080...
[Server] Access authorized. Monitoring command requests...
[Server] Access authorized. Monitoring command requests...
[Server] Upload complete. Saved encrypted bytes: neil.txt (15 bytes)
[Server] Encrypted stream delivered successfully for: neil.txt
[Server] Session terminated safely. Closing socket link.
[Server] Session terminated safely. Closing socket link.
```

- Shared_files showing logs

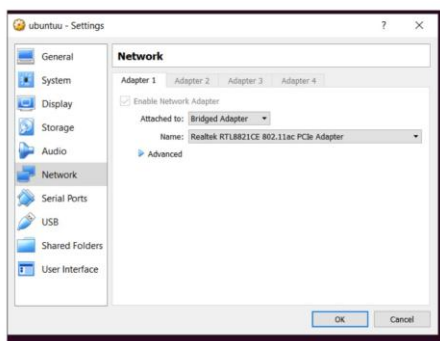


Distributed Network Setup

- Testing was done across two separate machines connected to PC's Wi-Fi hotspot.
- The server's IP address assigned by the hotspot was 192.168.182.86.

```
yvette@yvette-Latitude-7400-2-in-1:~/project_sys_net_prog$ ip route show | grep src
default via 192.168.182.63 dev wlo1 proto dhcp src 192.168.182.86 metric 600
192.168.182.0/24 dev wlo1 proto kernel scope link src 192.168.182.86 metric 600
```

- The gateway IP of the clients was changed to the dynamic IP address of the server and the server was made to listen for any open socket anywhere.
- The network settings were changed from NAT to **Bridged Adapter** using the physical wireless card (Realtek RTL8821CE). This placed the VM directly on the LAN so it could see the server.



- System running and client downloading a file.

```

neil@neil-VirtualBox:~/Desktop$ cd project_sys_net_prog
neil@neil-VirtualBox:~/Desktop/project_sys_net_prog$ ./file_client
[Client] Authentication Verification Complete. Session Secure.

=====
DISTRIBUTED FILE SYSTEM MENU
=====
1. Upload a File
2. Download a File
3. Exit System
-----
Enter your choice (1-3): 2
Enter filename to DOWNLOAD: neil.txt
[Client] Success! File downloaded completely and decrypted.
[Client] Verified Integrity Hash matching: 100A8480

=====
DISTRIBUTED FILE SYSTEM MENU
=====
1. Upload a File
2. Download a File
3. Exit System
-----
Enter your choice (1-3): 3
Exiting execution stream loop...
neil@neil-VirtualBox:~/Desktop/project_sys_net_prog$ cat neil.txt

Hi I am Neil
neil@neil-VirtualBox:~/Desktop/project_sys_net_prog$

```

- Client downloading a file that does not exist.

```

hi I am Neil
neil@neil-VirtualBox:~/Desktop/project_sys_net_prog$ ./file_client
[Client] Authentication Verification Complete. Session Secure.

=====
DISTRIBUTED FILE SYSTEM MENU
=====
1. Upload a File
2. Download a File
3. Exit System
-----
Enter your choice (1-3): 2
Enter filename to DOWNLOAD: holl.txt
[Client Error] File 'holl.txt' is absent from the storage server repository.

=====
DISTRIBUTED FILE SYSTEM MENU
=====
1. Upload a File
2. Download a File
3. Exit System
-----
Enter your choice (1-3):

```

- Client uploading a file.

```

neil@neil-VirtualBox:~/Desktop/project_sys_net_prog$ nano Aberline.txt
neil@neil-VirtualBox:~/Desktop/project_sys_net_prog$ ./file_client
[Client] Authentication Verification Complete. Session Secure.

=====
DISTRIBUTED FILE SYSTEM MENU
=====
1. Upload a File
2. Download a File
3. Exit System
-----
Enter your choice (1-3): 1
Enter local filename to UPLOAD: Aberline.txt
[Client] Server safely accepted stream and validated file structure bounds.

```

- Logs from the server showing the sequence of operations.

```

192.168.182.0/24 dev wlo1 proto kernel scope link src 192.168.182.00
yvette@yvette-Latitude-7480-2-in-1:~/project_sys_net_prog$ ./file_server
[Server] Secure environment running on port 8080...
[Server] Access authorized. Monitoring command requests...
[Server] Encrypted stream delivered successfully for: neil.txt
[Server] Access authorized. Monitoring command requests...
[Server] Encrypted stream delivered successfully for: neil.txt
[Server] Session terminated safely. Closing socket link.
[Server] Access authorized. Monitoring command requests...
[Server] File absent: holl.txt. Notifying client.
[Server] Session terminated safely. Closing socket link.
[Server] Access authorized. Monitoring command requests...
[Server] Upload complete. Saved encrypted bytes: Aberline.txt (24 bytes)

```

6. Analysis and discussion:

- Multiple client terminals could connect simultaneously hence multi-threading was achieved.
- Files were encrypted and decrypted over the air hence encryption was implemented successfully.
- Both endpoints compute identical verification hashes hence data integrity was maintained.
- Successful communication across separate machines over a pc's hotspot hence a distributed system as realized.

7. Challenges faced and solution

Mismatched Expected Hashes

Initially, the server tried to save temporary files inside a nested folder (shared_files). Because directory structures differed across the distributed machines, the server failed to read its own temporary file, resulting in an empty hash calculation.

Solution

Modified server.c to save the temporary file directly in the server's primary running folder (temp_dec_%d.txt), followed by an immediate unlink() call. This fixed the path issue and matched the hashes perfectly.

VirtualBox connection refused Error

The client VM could not connect to the server's IP initially because VirtualBox defaults to NAT mode, which isolates the VM from the host network.

Solution

Changed the VM network setting to Bridged Adapter and targeted the active Wi-Fi hardware. Running sudo dhclient forced the VM to grab a proper IP address from the phone hotspot, opening the connection.

8. Conclusions:

This lab demonstrated that developing applications for real networks introduces unique challenges compared to local testing. Hardcoded "localhost" addresses do not account for dynamic IP changes, virtual machine isolation, or network routing boundaries. Fixing these issues highlighted the practical steps needed to coordinate thread safety, file paths, and network configurations to build a secure distributed system.

8. References

- Linux manual
- Assistance from course mates